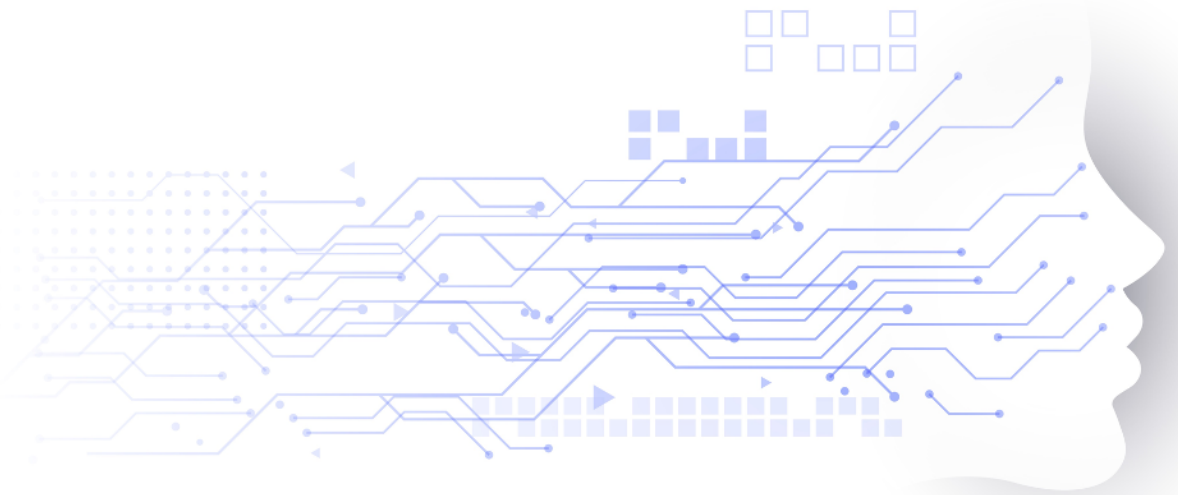
RMIT Classification: Trusted



Neural Networks

COSC 2673-2793

(Computational) Machine Learning

Image: Freepik.com

RMIT UNIVERSITY

Agenda

- Lecture videos:
  - Biologically inspired learning – Perceptron
  - Multi Layer perceptron
  - Back propagation
  - Practice issues
- Assignment 2 - discussion

Revision: Supervised Learning

In ***machine learning***, we wish to find the "best" hypothesis that approximates an unknown function:

$$h(\mathbf{x}) \approx f(\mathbf{x})$$

Experience: From which to learn (or find) the hypothesis

Task: Learn a model that to predict new "unseen" inputs.

Performance: Error measure of how closely the hypothesis predicts the target output

Algorithms:

- Regression, Logistics Regression, Regression Trees
- Decision Trees
- Rule Learning
- Ensembles, Random Forest

Feature Selection & Construction

Feature Selection allows a learner to use a subset of only the most relevant features of a problem to improve its performance. Method include:

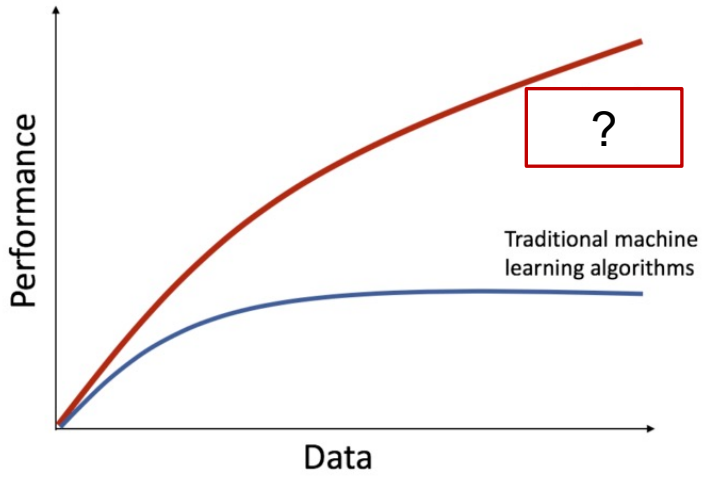
- Domain Expert Heuristics
- Filter methods, Wrapper methods, Embedded methods

Feature Construction is the process of generating additional features. The feature do not contain additional information, however, they may represent the information is a form that is easier for a leaner to use. Methods include:

- Projections & Kernel Functions
- PCA

Why do we need another method?

- Can we replicate how humans learn?
- Single model to handle a variety of tasks (regression-classification, vision-speech)
- Scale well with the size of experience and complexity of task.
- Efficient computations and parallelizable



Perceptron

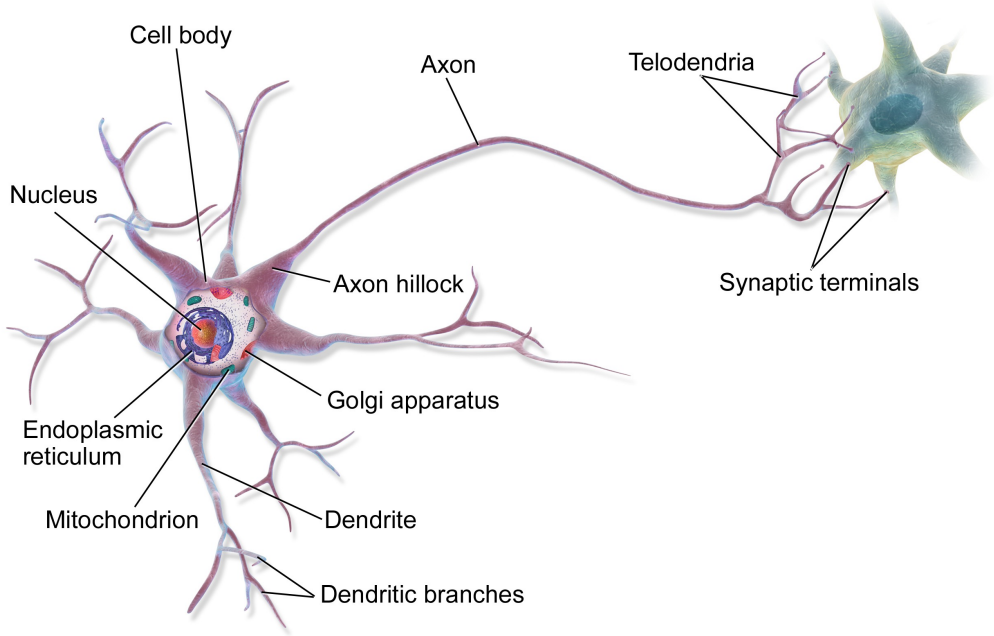
Biologically Inspired Learning

Neural Networks are inspired by the structure of the human brain

Many ML techniques are inspired by biology, or methods that humans and animals take for learning

A Neuron is formed of:

- A series of incoming synapses
- An activation cell
- A single outgoing synapse that connects to
- other Neurons



Neural Networks

Consists of many individual processing units (Neurons) with multiple connections (synapses) between them

Large number of Neurons: ~ 10^{10}

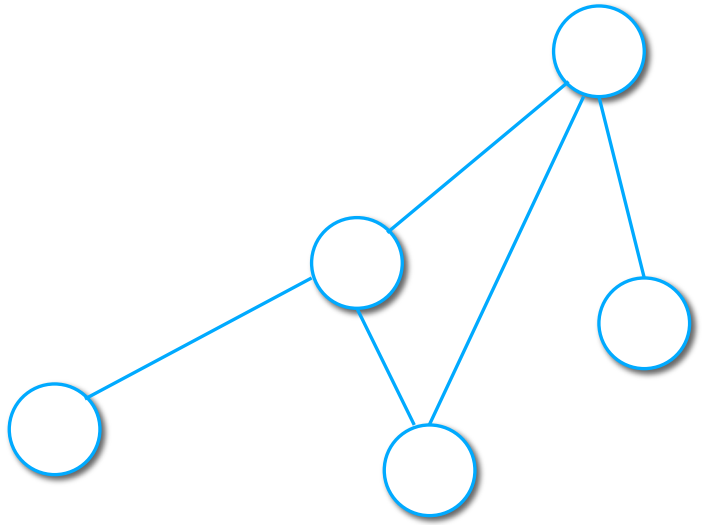
Large connectivity: ~ 10^5

Neurone switching time: ~ 0.001 s

Scene recognition time: ~ .1 s

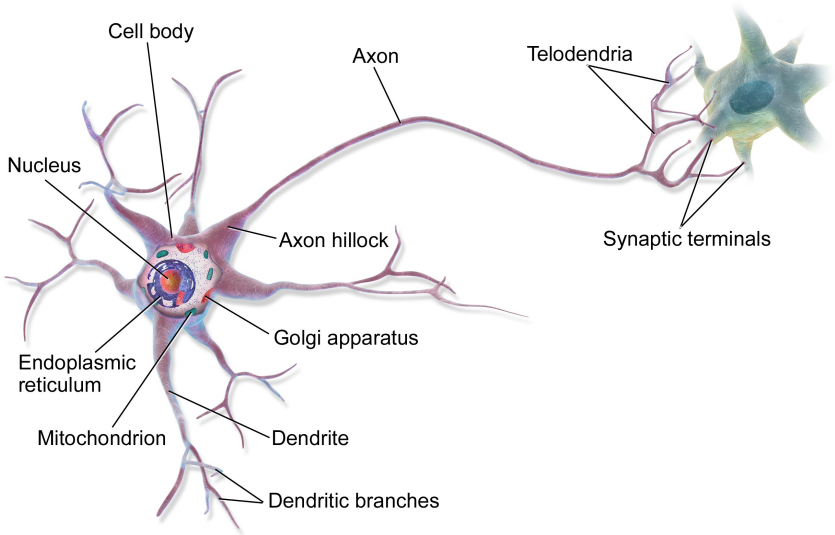
ANN:

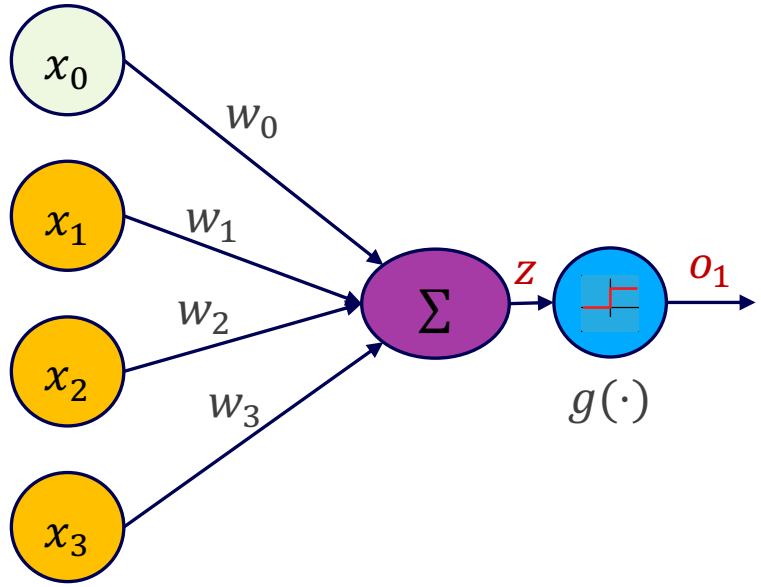
- Many neuron like threshold units
- Weighted interconnections between units
- Highly parallel processing



Perceptron

A Neuron is modelled as a ***Perceptron*** (Rosenblatt 1962):





$$z = w_0x_0 + w_1x_1 + w_2x_2 + \cdots$$

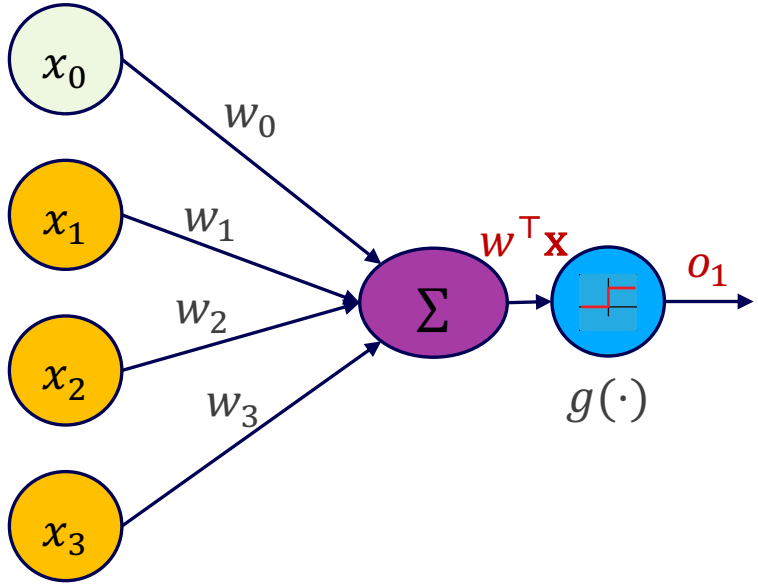
$$o_1 = g(w_0x_0 + w_1x_1 + w_2x_2 + \cdots)$$

Perceptron

A Neuron is modelled as a ***Perceptron*** (Rosenblatt 1962):

A Perceptron consists of:

- Multiple Input Connections: $\mathbf{x} = [x_0, x_1, \ldots, x_m]^\top$
- Bias (additional input): $x_0 = 1$
- Weights on each input: $\mathbf{w} = [w_0, w_1, \ldots, w_m]^\top$
- Activation Function: $g(\mathbf{w}^\top \mathbf{x})$
- Single output: o_1



Perceptron

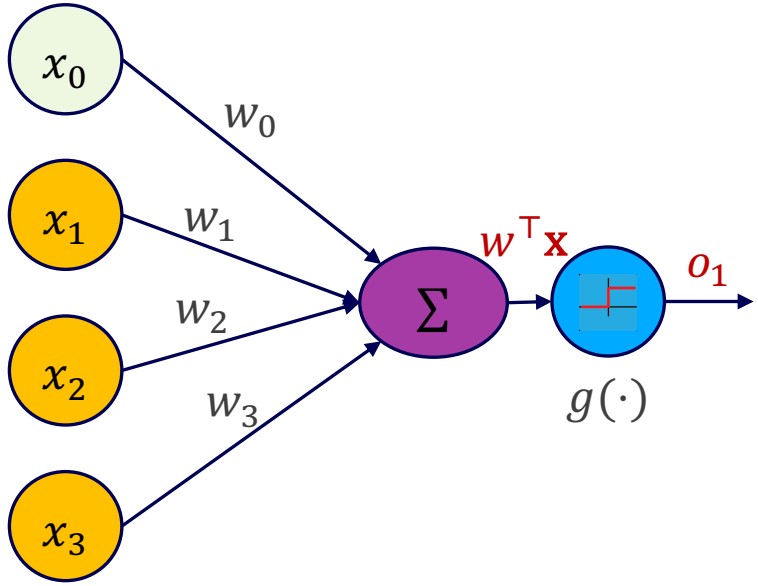
The output of the perceptron is calculated by:

Summing the products of each output and their weights

- $w_0x_0 + w_1x_1 + w_2x_2 + \cdots = \sum_{i=0}^{m} w_i x_i = \mathbf{w}^T\mathbf{x}$

Passing it through the activation function

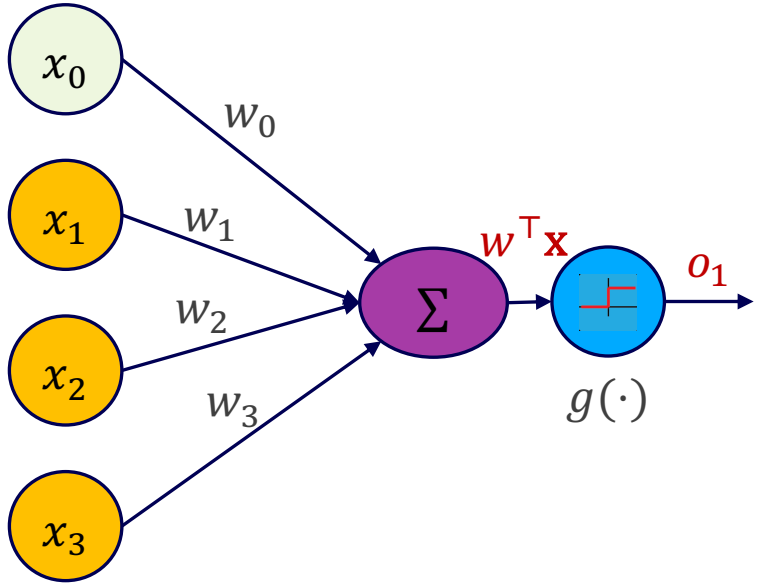
- A simple step function is used:
- $o_1 = g(\mathbf{w}^T\mathbf{x}) = \begin{cases} 1 & \text{if } \mathbf{w}^T\mathbf{x} \geq 0 \\ 0 & \text{if } \mathbf{w}^T\mathbf{x} < 0 \end{cases}$



Perceptron as a Binary Classifier

The perceptron can function as a binary classifier

- For example, two classes: $(+, -)$
- Output value of 1, means class $+$
- Output value of 0, means class $-$



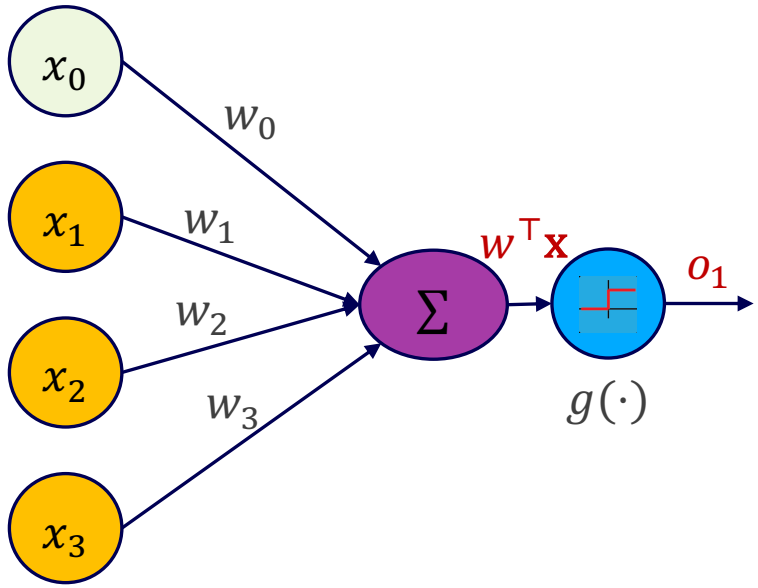
Learning with a Perceptron

Learning with a Perceptron involves finding values for the ***weights***:

- Observe that the calculation essentially linear regression!
  - $\sum_{i=0}^{m} w_i x_i = \mathbf{w}^T \mathbf{x}$
- Thus we could use gradient descent!

However, the step-function is discontinuous

- It cannot be differentiated



Learning with a Perceptron

Simple update rule for greedy algorithm:

1) Initialize the weights (random, all zero,)

2) Repeat until convergence

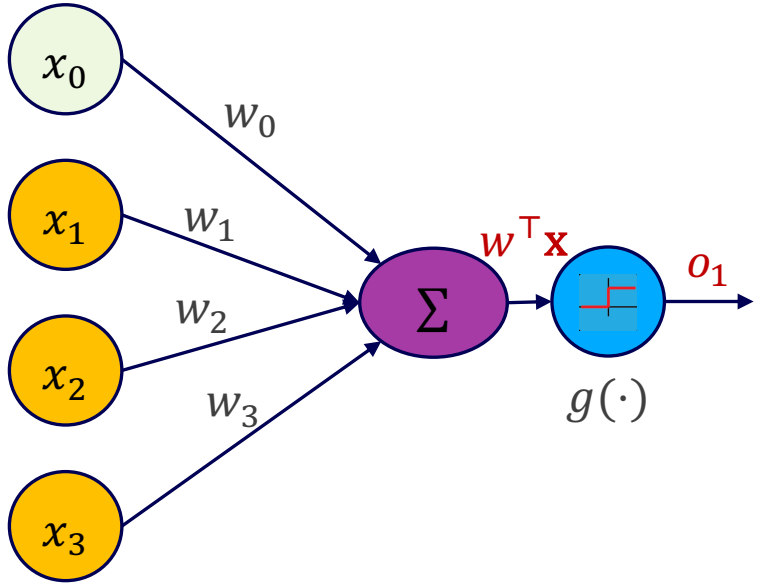
   For each example i in training dataset D

   - Calculate predicted output

$$o_1^{(i)} = g\left(\mathbf{w}^T \mathbf{x}^{(i)}\right)$$

   - Update the weight

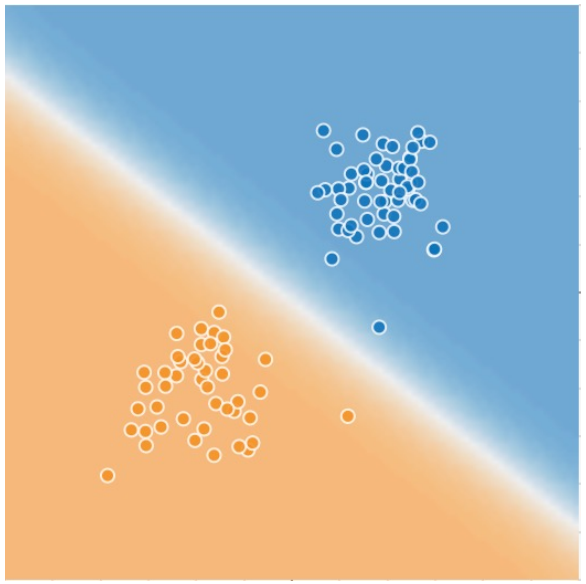
$$w_j = w_j + \alpha \left(y^{(i)} - o_1^{(i)}\right) x_j^{(i)}$$



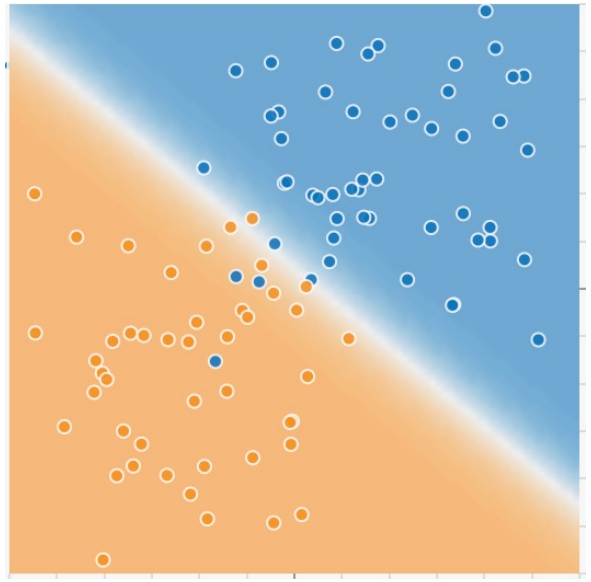
Also uses the ***learning rate***: $0 \leq \alpha \leq 1$

Learning with a Perceptron

- If training data is linearly separable then perceptron learning algorithm will converge to a solution.
- If data is not linearly separable, the algorithm fails and might even not result in an approximate solution.



linearly separable



Not linearly separable

Perceptron

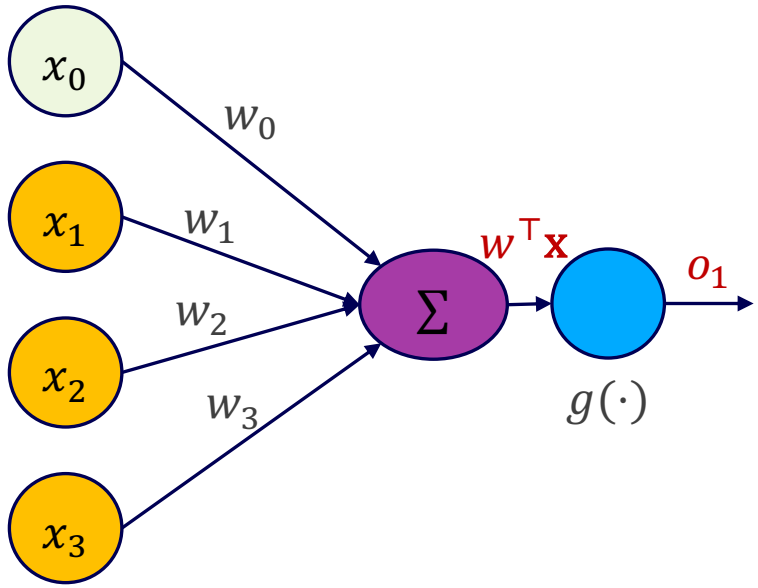
Perceptrons can model both **Regression** and **Classification**.

- This is dependent on the Activation Function
- For the purpose of this lecture, we will focus on Classification
  - This also more closely maps to the “activation” of a Neuron

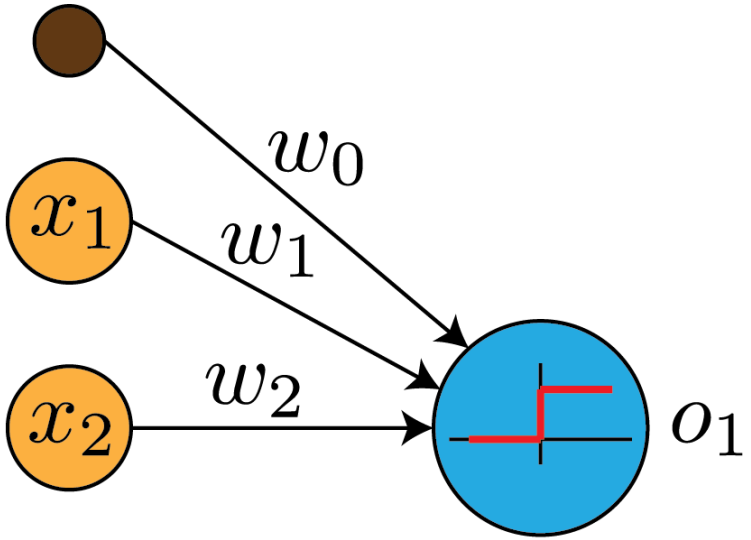
Observe that a Perceptron is essentially

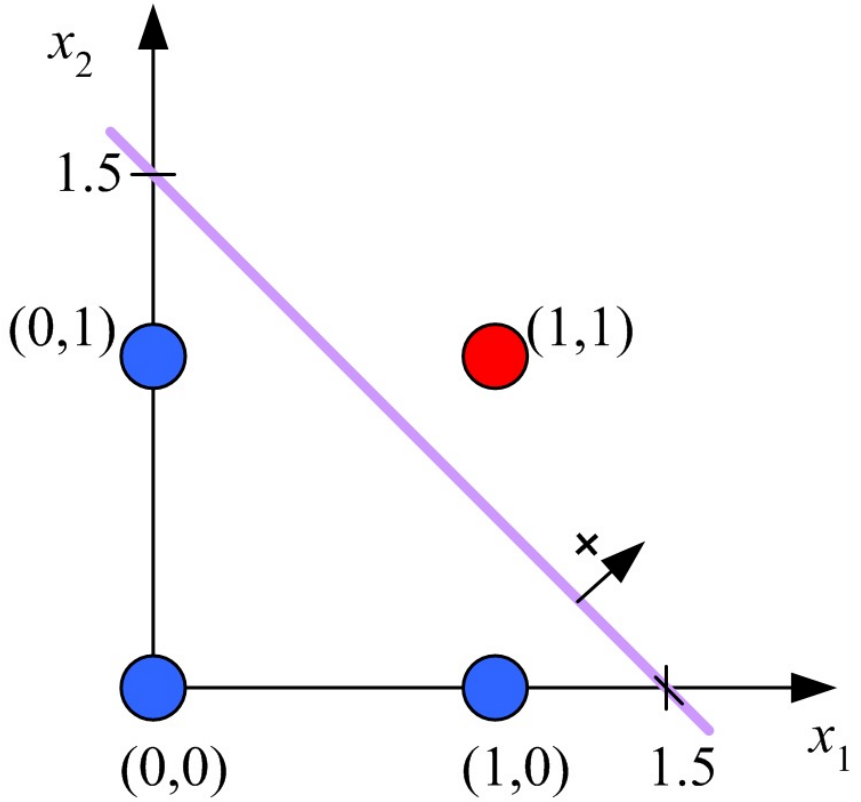
- Linear Regression
- Where the regression output is passed through the activation function!

$$g(\mathbf{w}^T\mathbf{x}) = \mathbf{w}^T\mathbf{x}$$

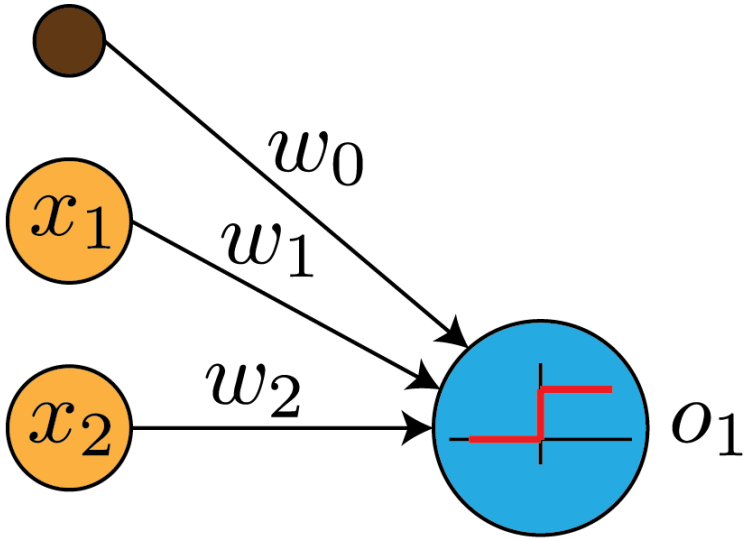


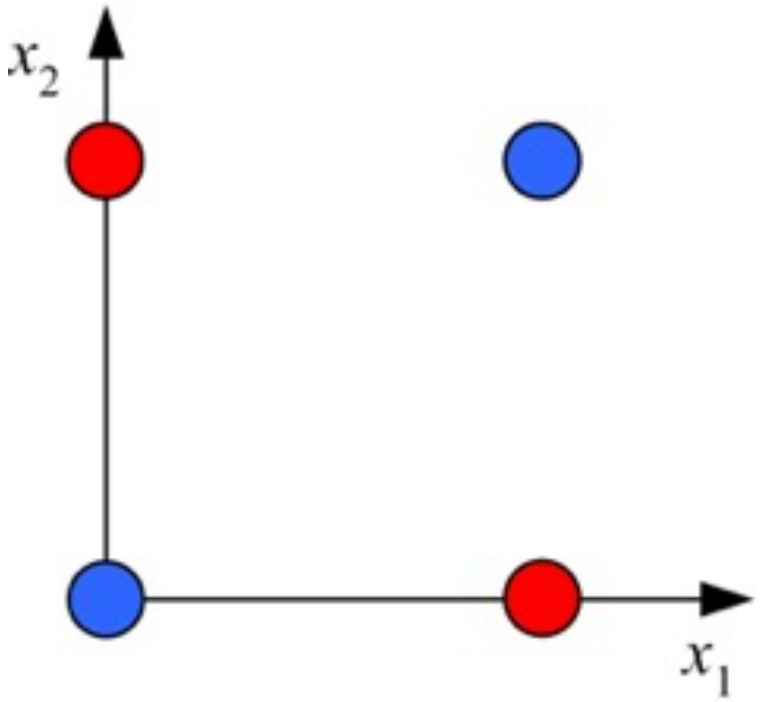
Boolean AND





XOR





(Minsky and Papert, 1969)

RMIT Classification: Trusted

Multi-Layer Perceptron

(MLP)

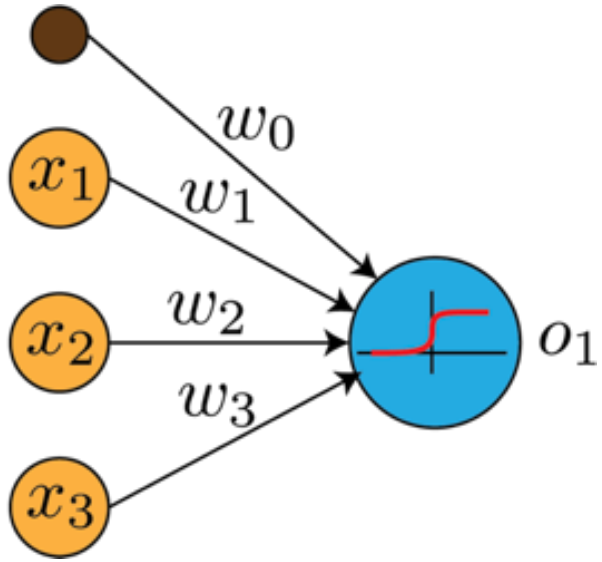
Learning with a Perceptron

We also observe that the basic perceptron is similar to ***logistics regression***!

Replace step function with ***sigmoid*** function

- $g(\mathbf{w}^T\mathbf{x}) = \frac{1}{1+e^{-\mathbf{w}^T\mathbf{x}}} = \sigma(\mathbf{w}^T\mathbf{x})$

Now, learning weights is similar to gradient descent



Revision: Logistic Regression Loss

We saw that the loss function (error) for logistics regression is given by the ***log-loss***:

$$J(\theta) = \frac{1}{n}\sum_{i=1}^{n} loss\left(h_\theta\left(\mathbf{x}^{(i)}\right), y^{(i)}\right)$$

$$J(\theta) = \frac{1}{n}\sum_{i=1}^{n} -y^{(i)}\log\left(h_\theta\left(\mathbf{x}^{(i)}\right)\right) - \left(1-y^{(i)}\right)\log\left(1-h_\theta\left(\mathbf{x}^{(i)}\right)\right)$$

Measure the error between

- Expected output $y^{(i)}$
- Actual output $h_\theta\left(\mathbf{x}^{(i)}\right)$

Find weights to: $\min_\theta J(\theta)$

In Perceptron (ANN) we represent θ_j as w_j

Revision: Logistic Regression Gradient Descent

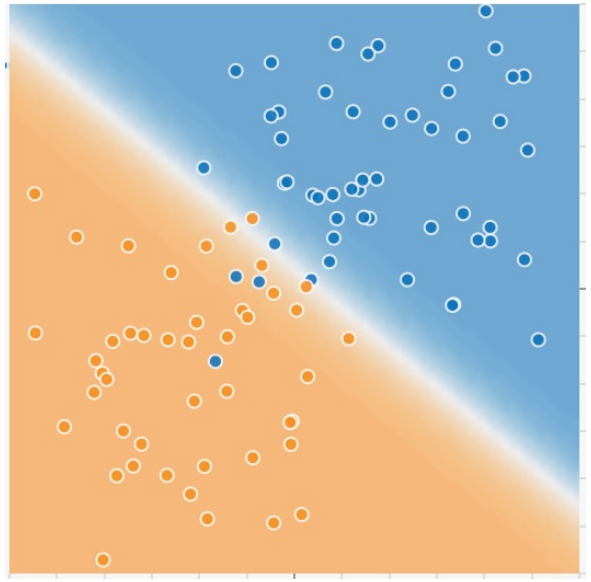
Given the loss function, gradient descent is:

$$\theta_j = \theta_j - \alpha \frac{\delta}{\delta \theta_j} J(\theta)$$

Where the partial derivatives are:

$$\frac{\delta}{\delta \theta_j} J(\theta) = \frac{1}{n} \sum_{i=1}^{n} \left(h_\theta \left(y^{(i)} \right) - \mathbf{x}^{(i)} \right) \mathbf{x}^{(i)}$$

Approximate solution when data is not linearly separable



Learning with Perceptron - Mini batch

For Neural Networks, the learning is similar, except

Use ***online learning***, where we don't calculate errors for *all* examples in the data set

Instead, we calculate the errors for a random ***batch*** of training example at a time

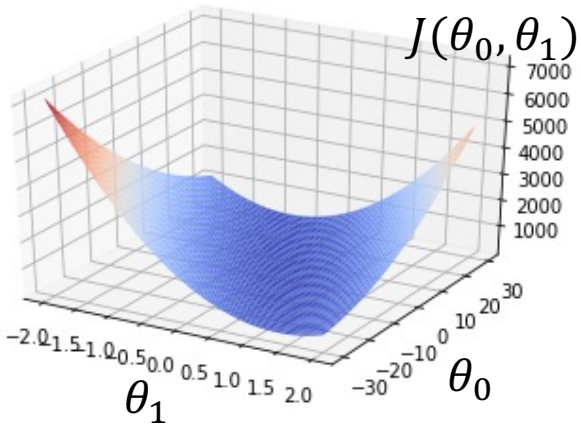
$$J(w) = \frac{1}{n}\sum_{i=1}^{n} loss\left(h_w\left(\mathbf{x}^{(i)}\right), y^{(i)}\right) = E_{x\sim p(x)}\left[loss\left(h_w\left(\mathbf{x}^{(i)}\right), y^{(i)}\right)\right]$$

$$E(\mathbf{w}) \equiv \frac{1}{b}\sum_{x^{(i)}\sim p(x)} loss\left(y^{(i)}, h_w\left(\mathbf{x}^{(i)}\right)\right)$$

We only use **b samples** to compute the gradient descent update

$$w_j = w_j - \alpha\frac{\delta}{\delta w_j}E(\mathbf{w})$$

Stochastic gradient decent



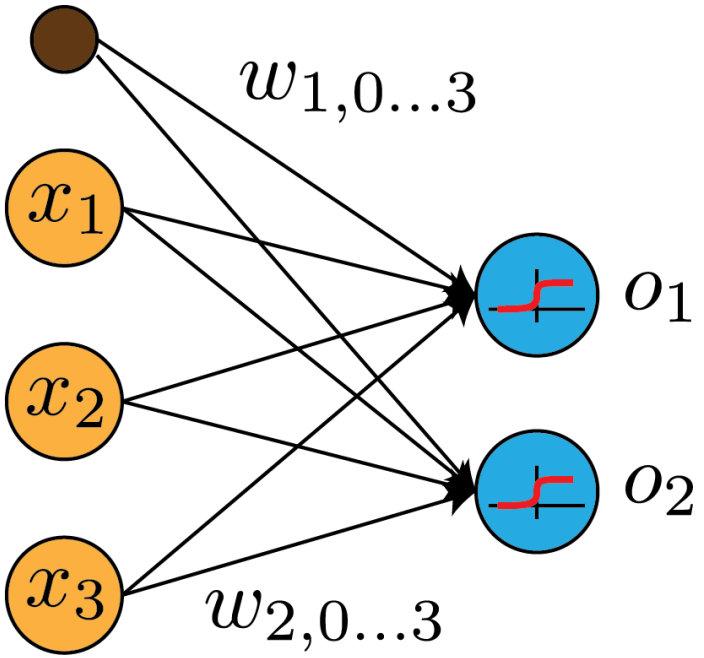
More than one Perceptron

A single perceptron is effectively just a single logistics regression equation

- For classification this is a binary classifier
- But what about being more general?

We can also represent a binary classifier using two perceptrons, connected to each output

- Each output is: $o_j = \sigma(\mathbf{w}_j^T \mathbf{x})$
- For class $+$, $o_1 = 1, o_2 = 0$
- Vice-versa for class $-$, $o_1 = 0, o_2 = 1$
- **To classify choose**: $\max_j o_j$

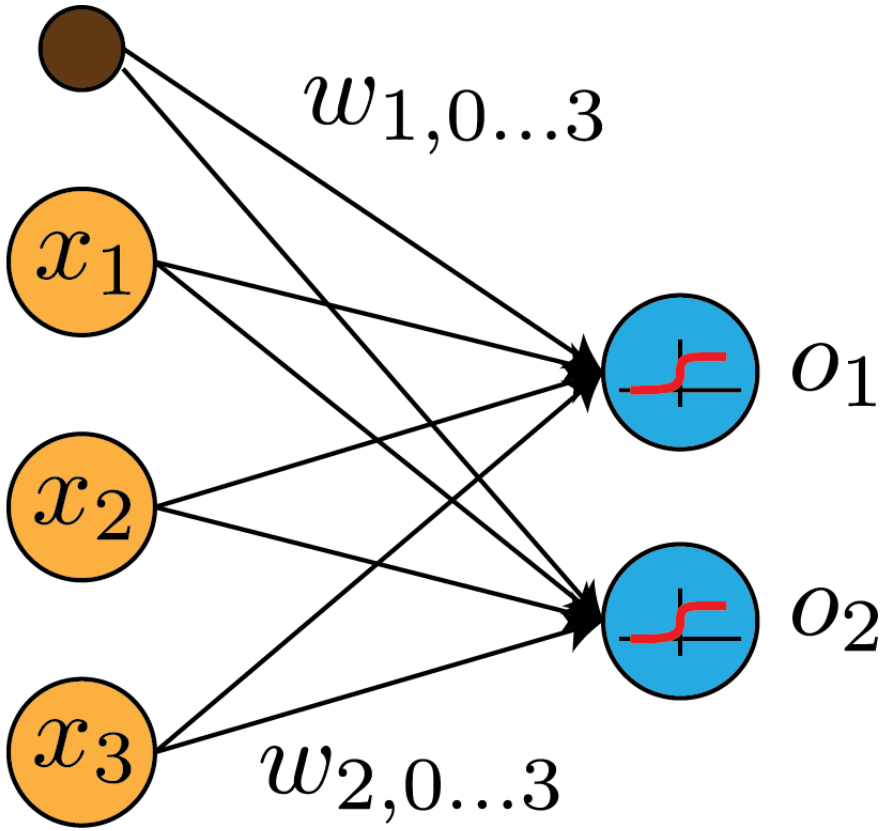


More than one Perceptron

Training is identical to with a single perceptron

$$\mathrm{w}^\top = \begin{bmatrix} w_{1,0} & w_{1,1} & w_{1,2} & w_{1,3} \\ w_{2,0} & w_{2,1} & w_{2,2} & w_{2,3} \end{bmatrix} = \begin{bmatrix} w_1^\top \\ w_2^\top \end{bmatrix}$$

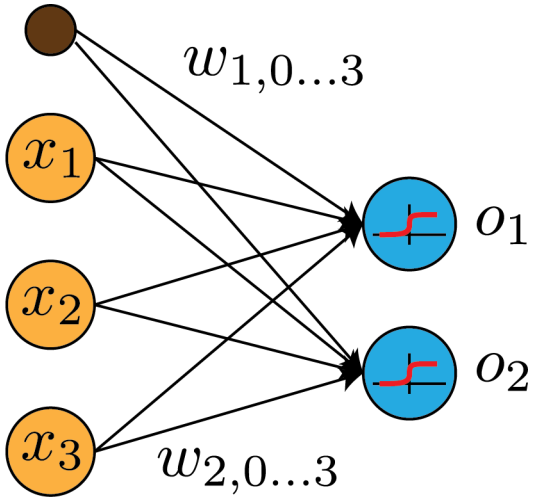
$$\begin{bmatrix} o_1 \\ o_2 \end{bmatrix} = \sigma\left(\begin{bmatrix} w_{1,0} & w_{1,1} & w_{1,2} & w_{1,3} \\ w_{2,0} & w_{2,1} & w_{2,2} & w_{2,3} \end{bmatrix} \begin{bmatrix} 1 \\ x_1 \\ x_2 \\ x_3 \end{bmatrix} \right)$$

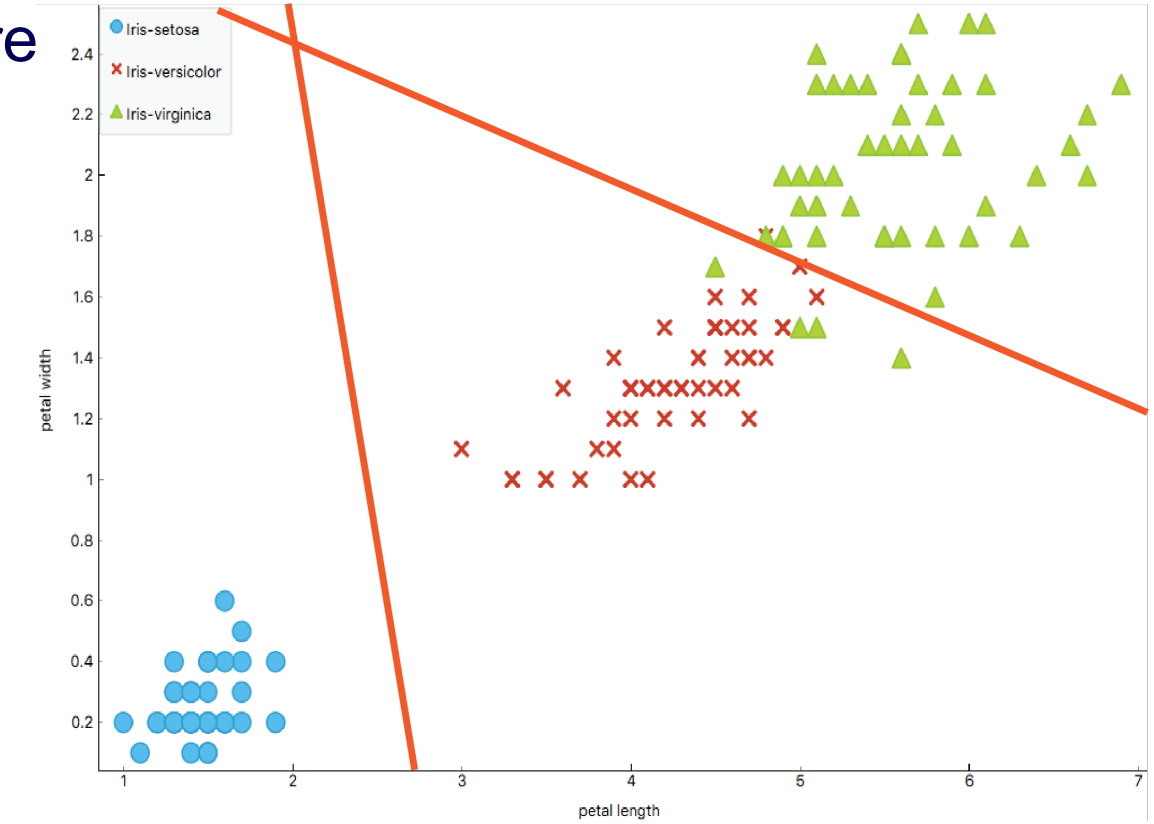


More than one Perceptron

Extending to multiple classes requires simply adding additional perceptrons, one for each class

Of course, this gives us nothing more than multiple logistics regression functions



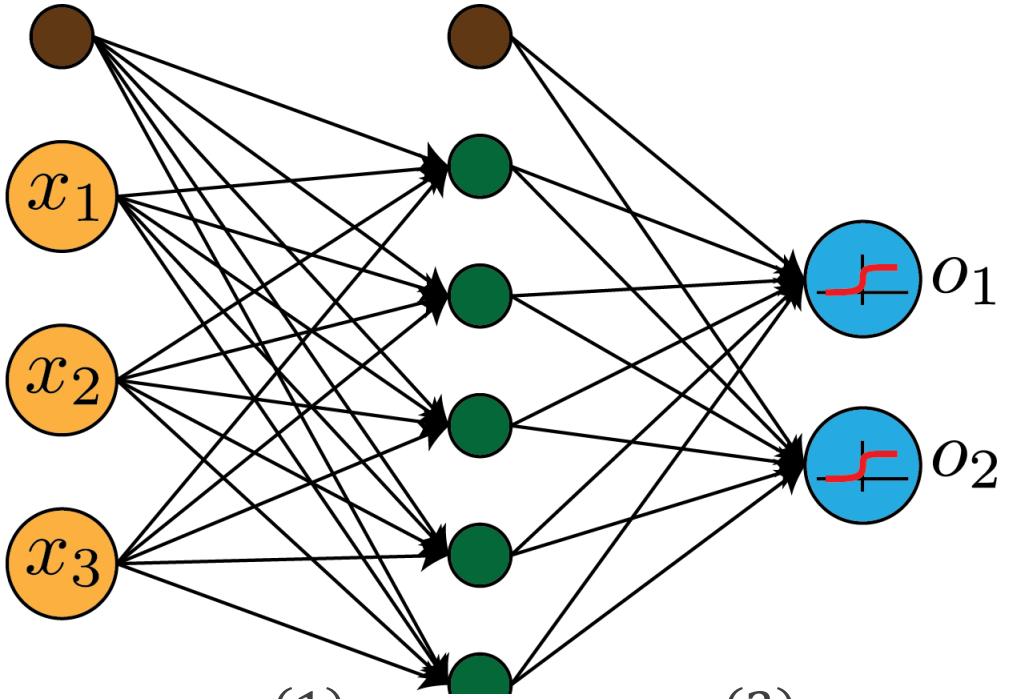


Multi-Layer Perceptron (MLP)

A multi-layer perceptron, introduces a ***hidden layer*** of perceptrons

- Each hidden layer perceptron has a sigmoid function
- The perceptrons are fully connected
- Output of the network is computed using a feed-forward method
  - $h_k = \sigma\left(\left(w_k^{(1)}\right)^{\mathsf{T}} \mathbf{x}\right)$
  - $o_j = \sigma\left(\left(w_j^{(2)}\right)^{\mathsf{T}} \boldsymbol{h}\right)$

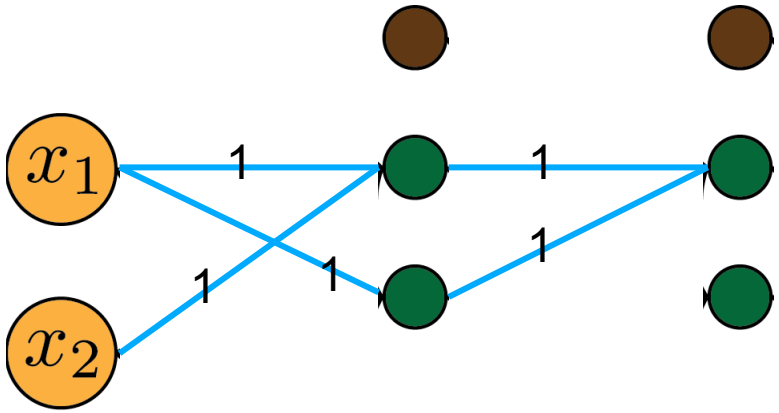
This may also be known as an ***Artificial Neural Network (ANN)***



$$\boldsymbol{h} = [h_k]_{k=0}^{m}$$

Multi-Layer Perceptron (MLP)

Importance of non-linear activations



RMIT Classification: Trusted

Training an MLP

Back-Propagation

Training a MLP

How can an MLP be trained?

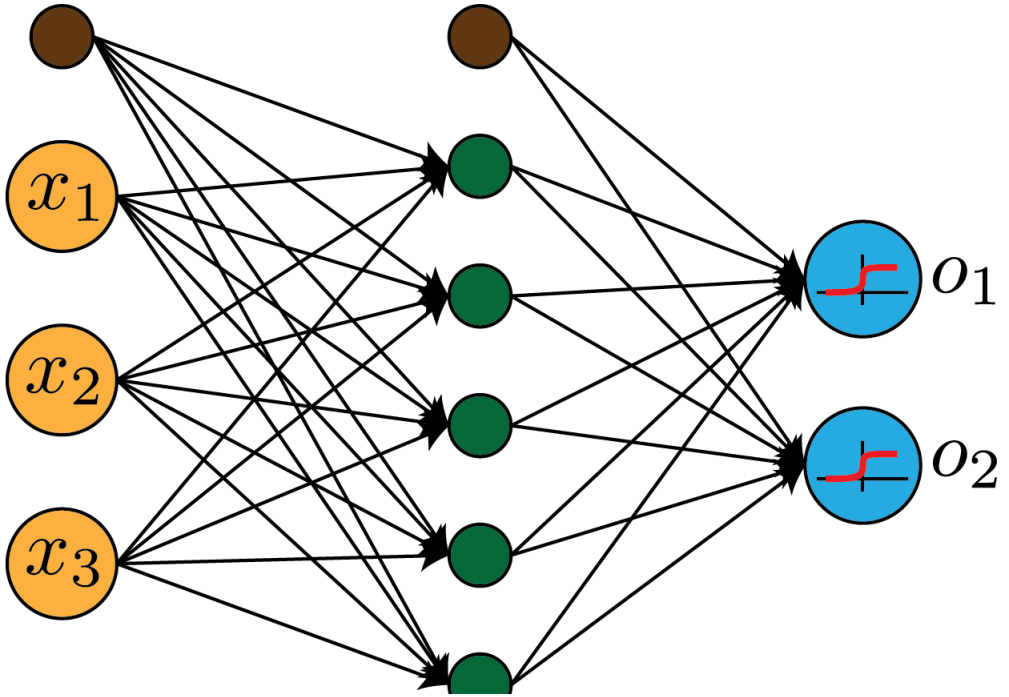
- How can we adapt gradient descent?

If we consider just the output layer

- Same as before
- Loss function (error) is still between the expected output and actual output

$$w_j^{(2)} = w_j^{(2)} + \alpha \sum_i \left(y^{(i)} - o_j^{(i)} \right) h_k^{(i)}$$

  - Except use the outputs of the hidden layer



Training an MLP

How can we train the weights of the hidden layer?

$$w_j^{(1)} = w_j^{(1)} + \alpha \sum \left(?^{(i)} - h_k^{(i)} \right) x_l^{(i)}$$

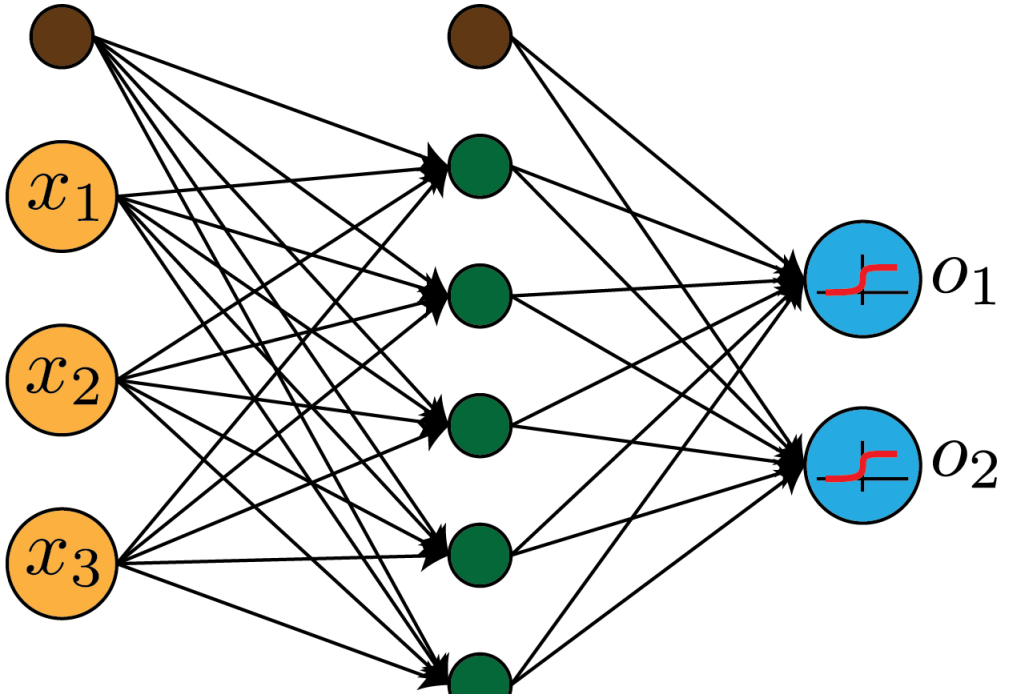
We have the input component

BUT! We don't have the "expected output"

Or do we?

We want to ***distribute*** the error backwards through the network to the earlier layers

This is known as ***back-propagation***



Derivatives

- Analytic derivative

$$f(x, y, z) = (x + y)z \longrightarrow \frac{\partial f(x, y, z)}{\partial x} = z$$

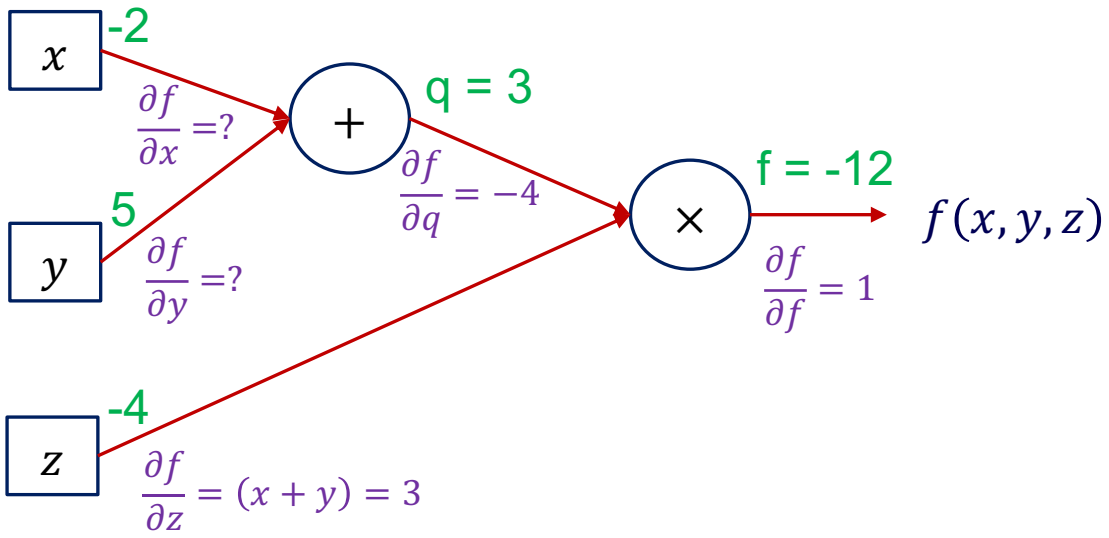
- Numeric derivative

$$f(x, y, z) = (x + y)z \longrightarrow \frac{\partial f(x, y, z)}{\partial x} = \lim_{\Delta \to 0} \frac{f(x + \Delta, y, z) - f(x, y, z)}{\Delta}$$

$$\frac{(x + \Delta + y)z - (x + y)z}{\Delta}$$

Back-Propagation

$$f(x, y, z) = (x + y)z$$

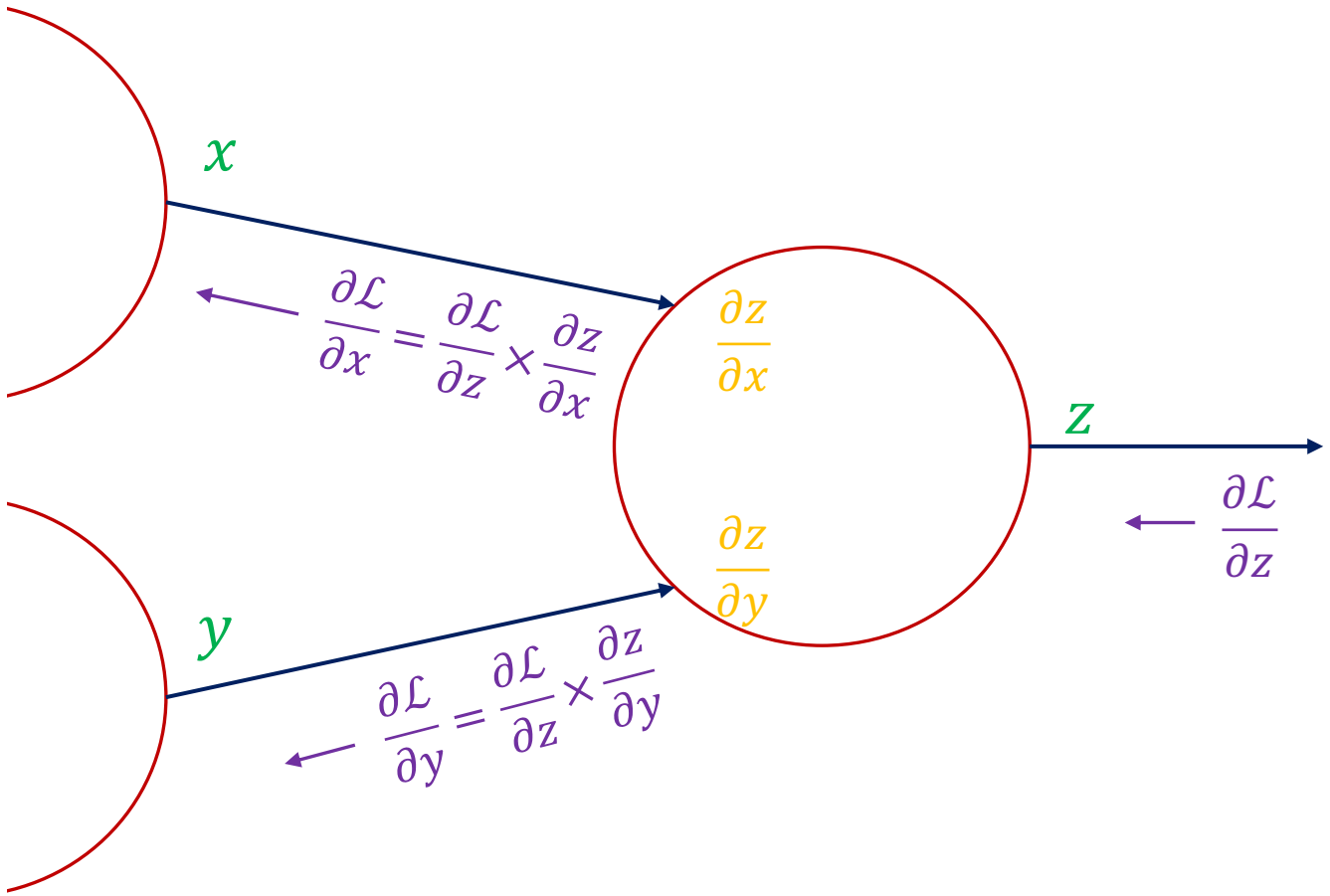


$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q}\frac{\partial q}{\partial x} = -4 \times 1$$

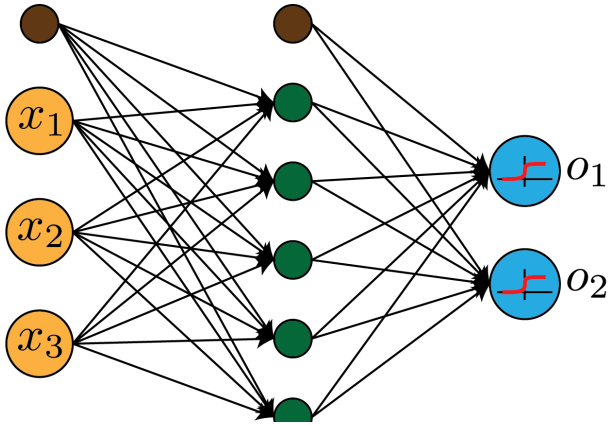
$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q}\frac{\partial q}{\partial y} = -4 \times 1$$

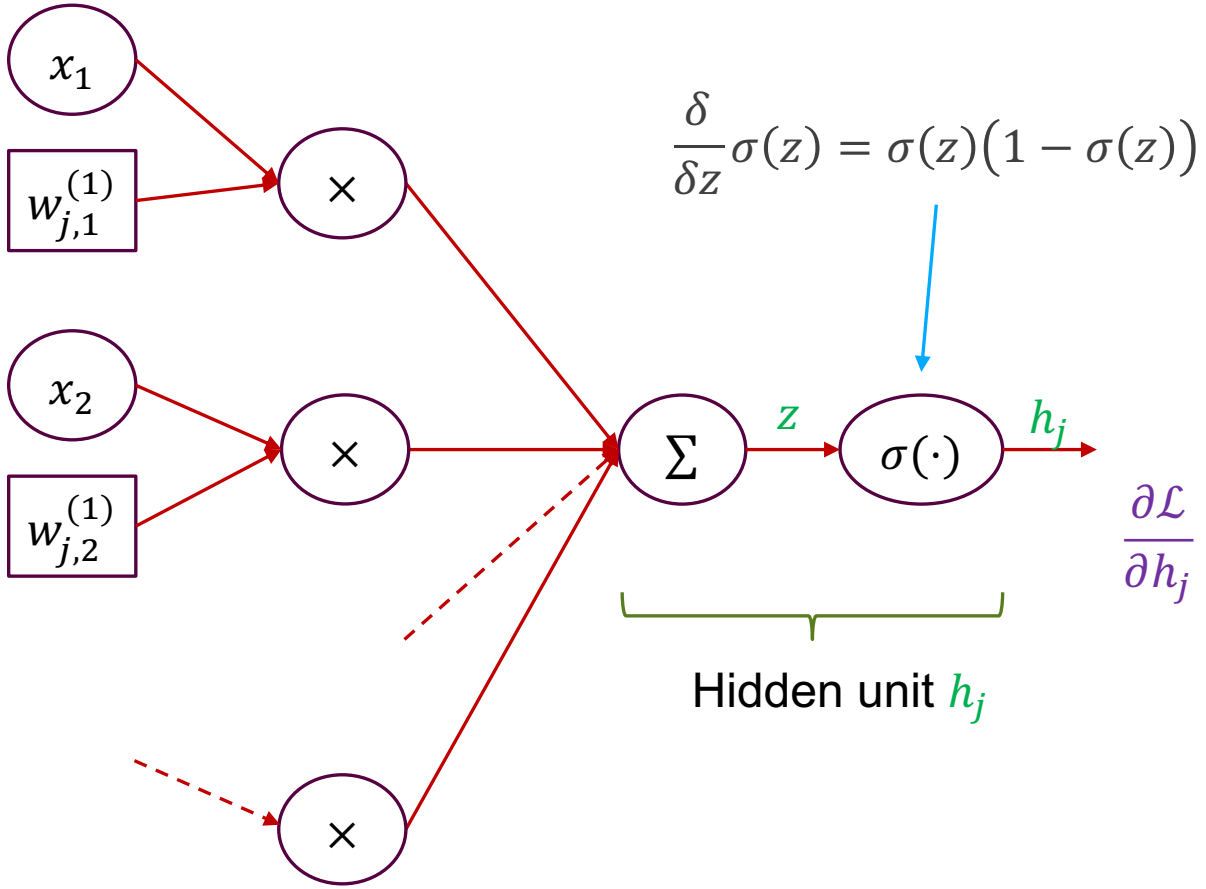
https://cs231n.github.io/optimization-2/

Back-Propagation



Back-Propagation via the Chain Rule





Training Process

The MLP can be trained by:

Online training, where the weights are updated after each training instance

- Smaller distribution of weights
- Slower training, as weights are adjusted more often

Batch training, where the error is accumulated across multiple instances

- A batch is not necessarily the entire data set
- Weights update slower and may generally require more epochs

For an MLP, as the updates are gradual, multiple passes of the data set is required.

Each pass is an ***epoch***

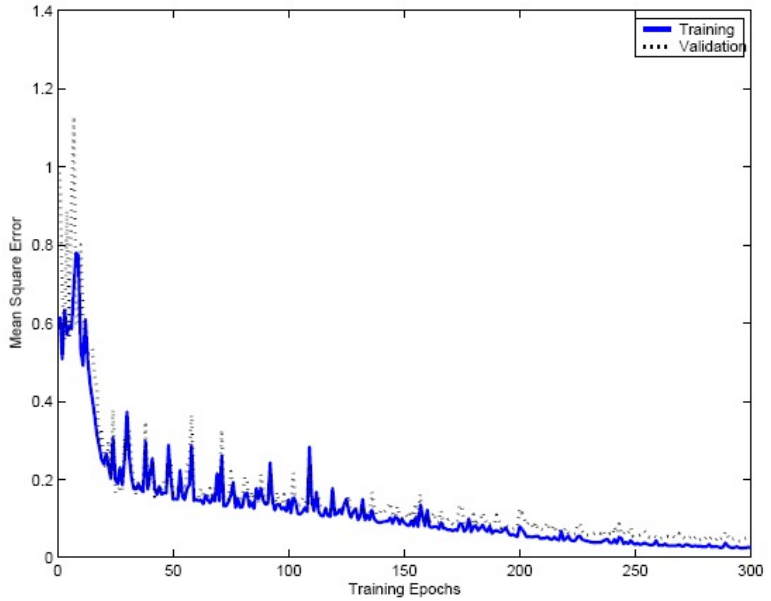
Most algorithms are batch mode, with a tuneable batch size

Training Process

The training algorithm (regardless of mode is):

1. Initialise all weights to small non-zero random values
2. For each training instance (or batch)
    a. Processes the instances feed-forward through the network,
    b. Calculate the error of the batch
    c. Update the network weights, with a feed-forward instance
3. Repeat Until Convergence

Example Training



The mean square error on training and validation sets as a function of training epochs

RMIT Classification: Trusted

MLP Learning

interpretation

MLP Learning

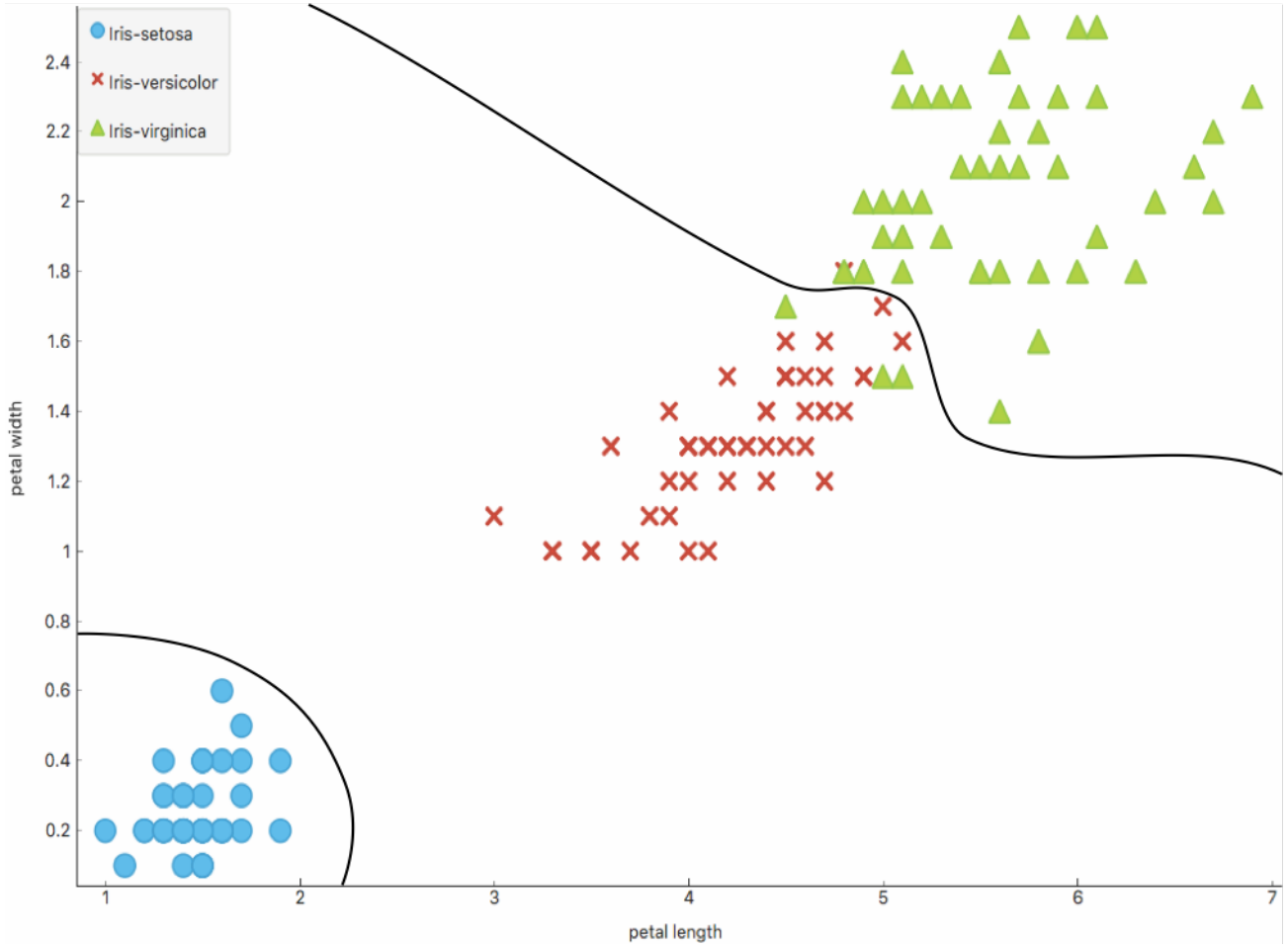
MLPs are complex and it is very difficult to follow how a data example is processed.

We can interpret and view the learning of a MLP in two ways

- Nature of the Hypothesis
- Intent of the hidden layer

Nature of an MLP Hypothesis

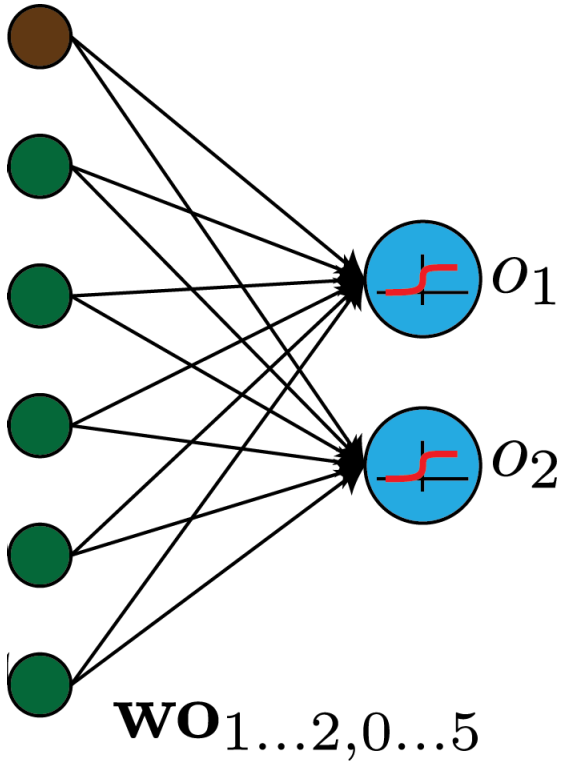
The multiple layers of the MLP transform the linear logistics equation into a complex non-polynomial hypothesis



Intent of the Hidden Layer

If we isolate the hidden layer to the output layer of the MLP

- The hidden layer acts as the attributes/features of the output nodes
- That is, the hidden layer form a state-space



Intent of the Hidden Layer

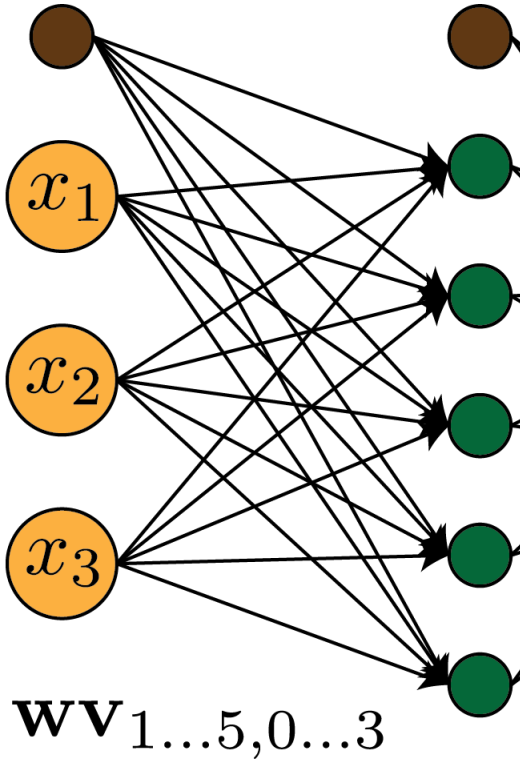
So what does the input to hidden-layer do?

One view is that it transforms the original feature space into a new feature space

That is, it acts as a kernel function

Indeed, for complex neural networks (CNNs) it is possible to "view" what the hidden layer(s) have learnt

- Or at least visualise their activations



$$K\left(\mathbf{x}, W^{(1)}\right) = \sigma(W^{(1)}\mathbf{x})$$

Strengths of MLPs

- Every bounded continuous function can be approximated with arbitrary small error by network with one hidden layer
  - [Cybenko 1989], [Hornik et. al. 1989]
- Any function can be approximated to arbitrary accuracy by a network with two hidden layers
  - [Cybenko 1989]

Strengths of MLPs

MLPs have a number of strengths compared to other ML techniques:

- Represent various ML problems including classification & regressions
- Generate models with complex non-linear functions
- Do not necessarily require manual data pre-processing
- “Discover” alternative features/attributes for representing a ML problem

RMIT Classification: Trusted

Practice Issues

Convergence

Network design

Over-fitting

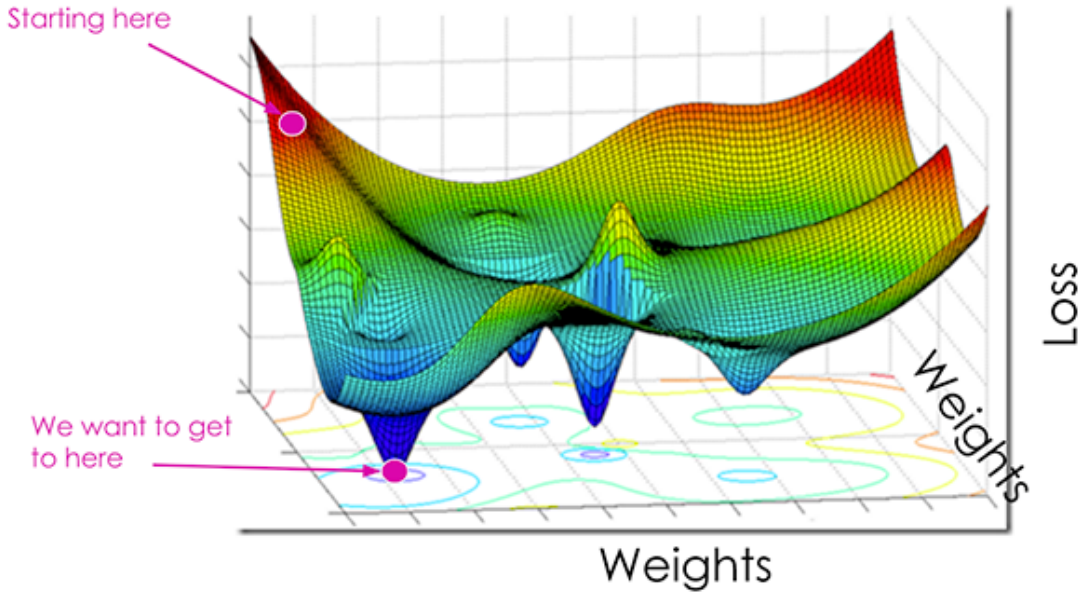
Issues in MLPs

- Convergence of Back-propagation
- Network design
  - Size of the hidden layer & number of free parameters
  - Network Complexity
- Over-fitting

Convergence

The back-propagation algorithm relies on gradient descent.

Recall from regression that gradient descent may get stuck in local minima



Convergence

The back-propagation algorithm relies on gradient descent.

- Recall from regression that gradient descent may get stuck in local minima

Thus Back-propagation has the same limitation

- It is not guaranteed to ***converge*** to the optimal minimum error
- However, *in practice*, MLPs do converge to a suitable error
  - The noisy gradients via stochastic gradient calculation helps.

$$E(\mathbf{w}) \equiv \frac{1}{b} \sum_{x^{(i)} \sim p(x)} loss\left(y^{(i)}, h_w\left(\mathbf{x}^{(i)}\right)\right)$$

Momentum

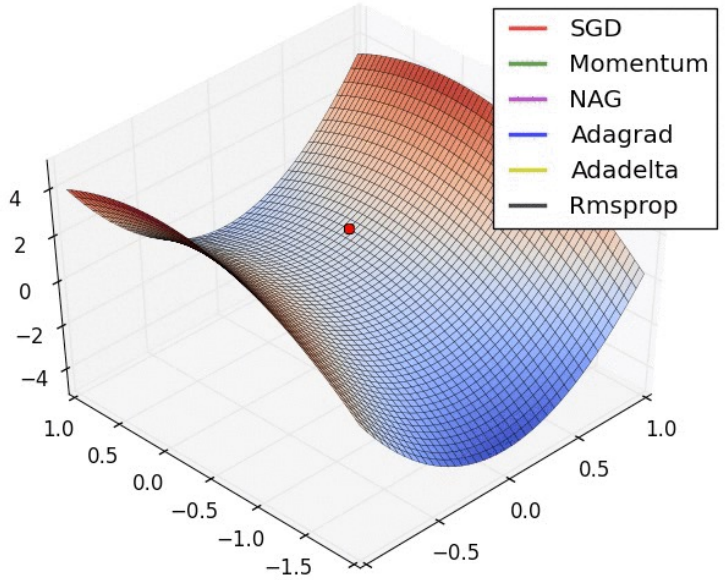
Momentum

Add an additional momentum term when adjusting weights

$$v_t = \gamma v_{t-1} + \alpha \nabla_{w_t} E[w_t]$$

Learning Rate: α $w_t = w_{t-1} - v_t$

Momentum: γ



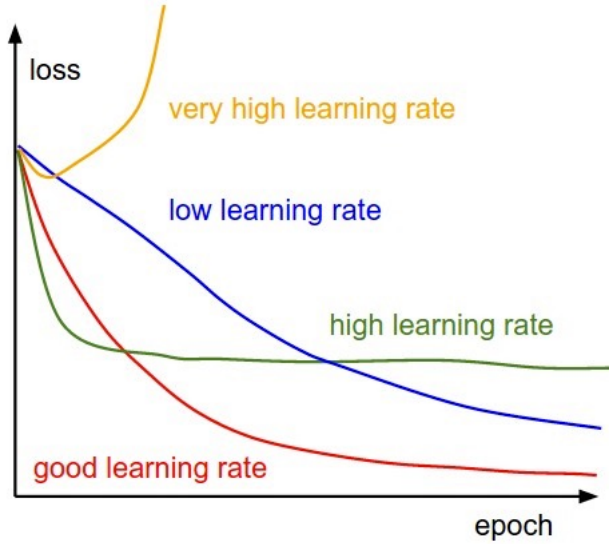
Images credit: Alec Radford.

Demo: https://distill.pub/2017/momentum/

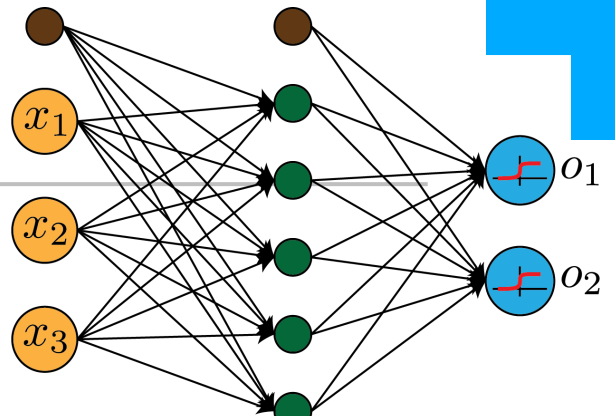
Adaptive Learning Rate

Adaptive Learning Rate

- Dynamically adjust the learning weight
- If the training error decreases -> increase the learning rate to converge faster
- If the error increases, slow down learning



Network Design



A key challenge of using MLPs is designing the network structure:

The number of nodes in the hidden layer

The form of the target output

- Classification problem(s) - with the encoding the of the classification
- Regression problem(s) - number of output nodes and the expected value

Adding more nodes:

Makes the network more complex and improves the internal representation

Potentially Requires more training time

Distributes the error(s) across more nodes

Can also make a network too complex, leading to high variance and overfitting

For too many weights, the network will not generalise well

VC-Dimensions

Intuitively, adding more nodes (and thus weights) increases the complexity of the MLP

- Increases the hypothesis space
- Increases the VC-Dimension of the MLP

PAC-learning gives insight to the complexity of an MLP

- The VC-Dimension of a network of *perceptrons* (with step-functions) I

$$VC(G_{perceptrons}) \leq 2(r+1)s\log(es)$$

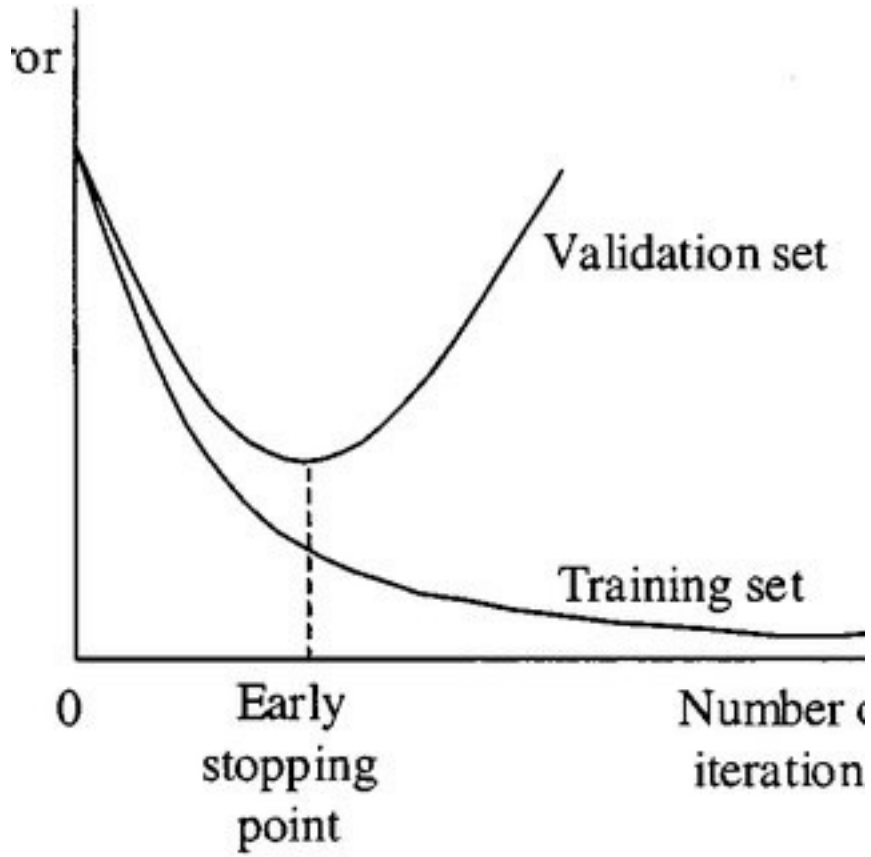
- For s perceptrons with r inputs. e is the base of the natural logarithm
- The VC-Dim is order $O(n\log n)$ is the number of perceptrons

An MLP with the sigmoid is at least as complex as this

RMIT Classification: Trusted

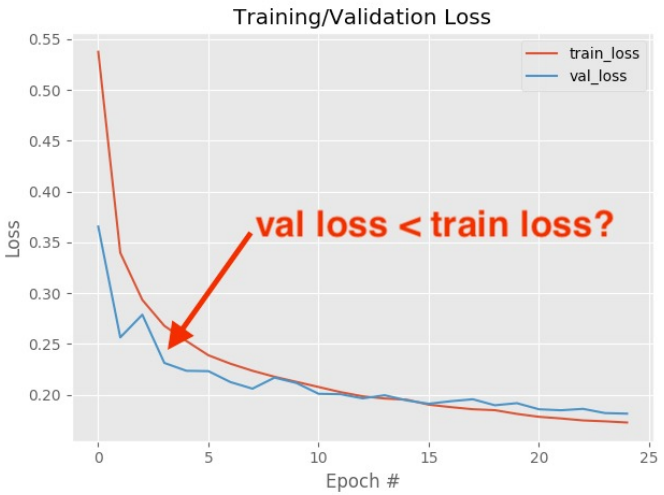
Over-Fitting

- **Early stopping**
- Regularization
- Dropout



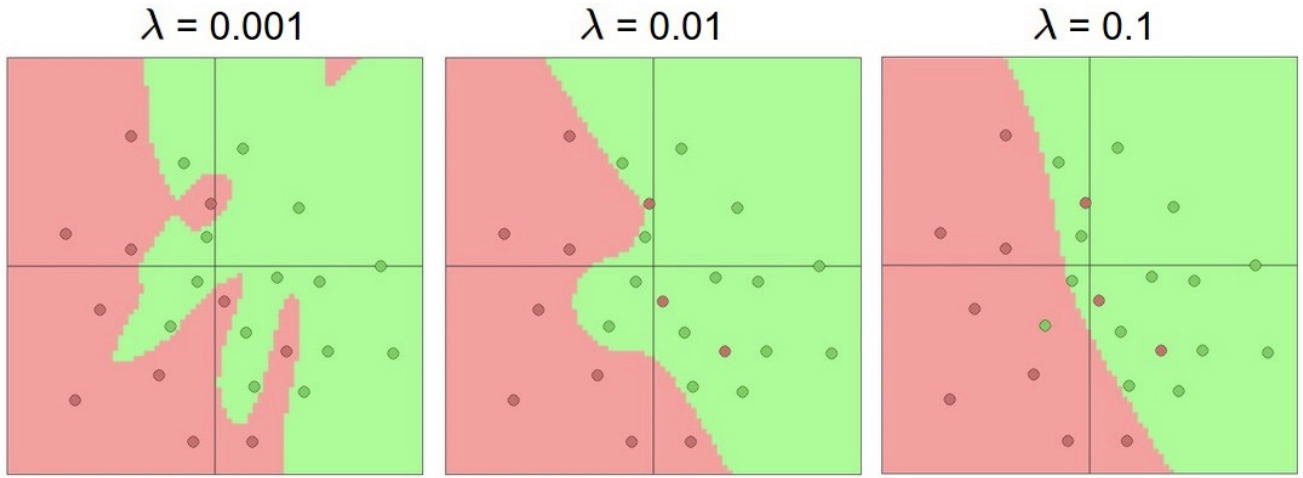
Can validation loss be smaller that training loss?

- Regularization applied during training, but not during validation/testing
- Training loss is measured *during* each epoch while validation loss is measured *after* each epoch
- ***The validation set may be easier than the training set (or there may be leaks)***



Over-Fitting

- Regularization
  - Lasso penalty - $\sum_{i,j}|w_{i,j}|$
  - Ridge ridge penalty - $\sum_{i,j}(w_{i,j})^2$



Demo: https://cs.stanford.edu/people/karpathy/convnetjs/demo/classify2d.html

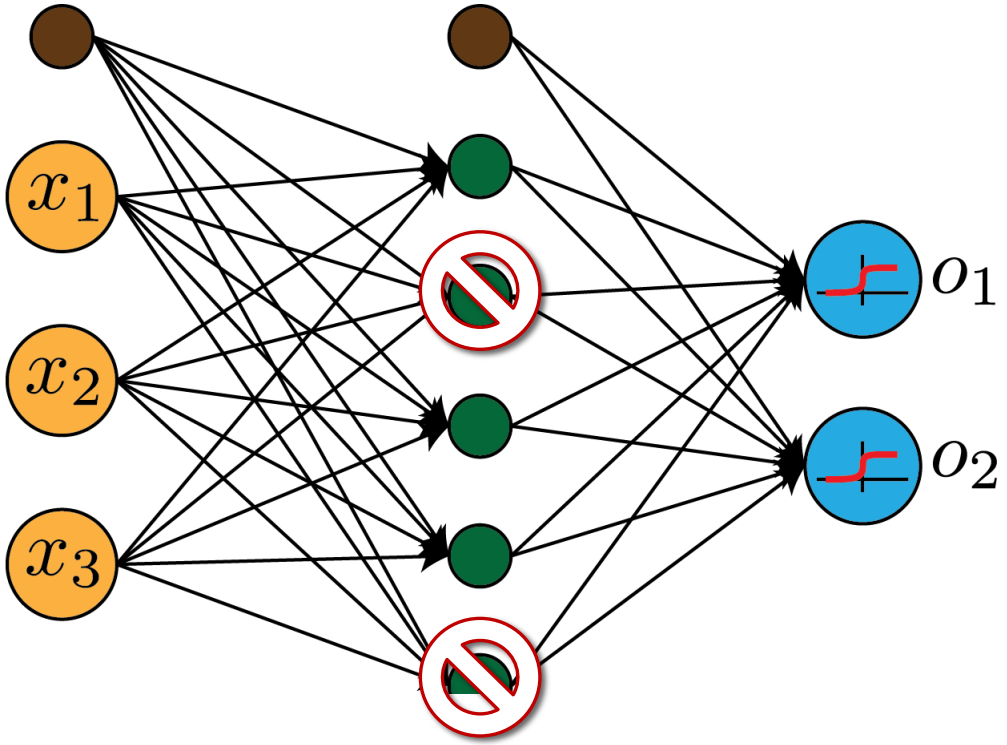
Dropout

A fully connected MLP is ***symmetric***

- Each perceptron of the network is identical
- Back-propagation, is theoretically sound, but with symmetry, in practice learning is slower and less accurate

Dropout randomly removes synapses (connections) between perceptrons in the MLP

- Forces a non-symmetric network
- Forces the network to consider different combinations of the input attributes
- Cannot be over reliant on a single unit



Hyper parameter tuning

The most common hyperparameters in context of Neural Networks include:

- The initial learning rate
- Learning rate decay schedule (such as the decay constant)
- Regularization strength (L2 penalty, dropout strength)

- Mostly done with hold-out validation (cross-validation may not be realistic with deep networks)

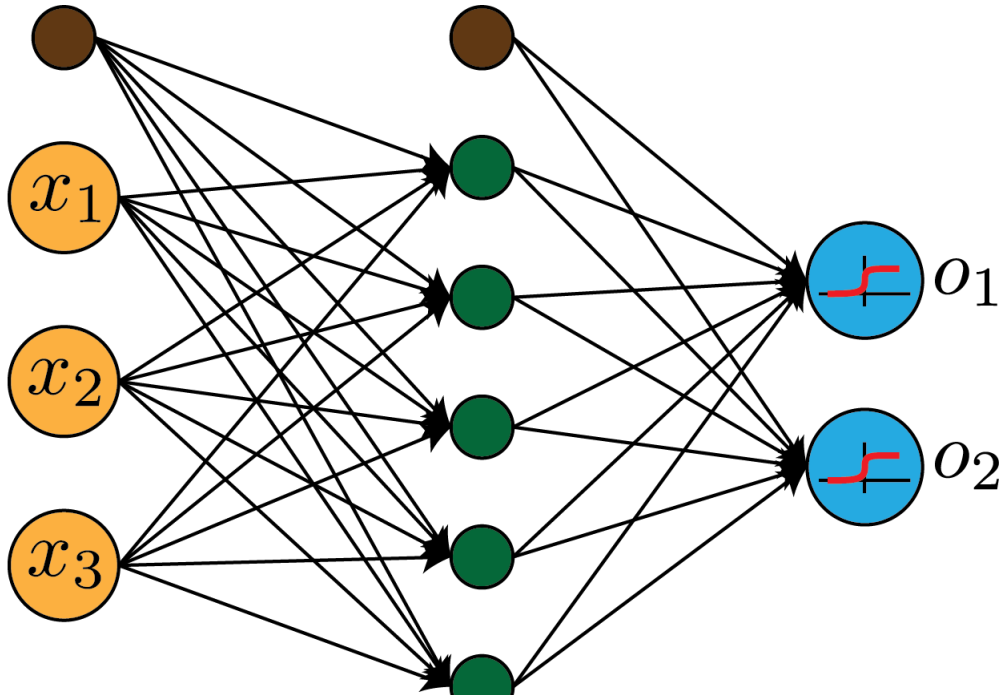
Random search or grid search: http://www.jmlr.org/papers/volume13/bergstra12a/bergstra12a.pdf

Weight Initialization

A fully connected MLP is ***symmetric***

Each perceptron of the network is identical

Weights need to be initialized to random values.



Multiple Hidden Layers

Additional Hidden Layers can be added to the network:

- Adding hidden layers can increase the complexity of a MLPs hypothesis and what it can learn at a faster rate than adding additional perceptrons into a single hidden layer
- Produces a simpler & smaller network that has fewer weights to train
- Back propagation is continued through the chain rule
  - However, the errors become more and more diluted as layers are added

MLP with one hidden layer is a universal approximator (Hornik et al., 1989), but using multiple layers may lead to simpler networks

- The multi-layer network can compute a hierarchy of new “features/attributes”
- Generalises and enriches the feature representation(s) before logistics classification

Demo

http://playground.tensorflow.org

RMIT Classification: Trusted

Dimensionality Reduction

Dimensionality Reduction

A layer of an MLP does not need to have more perceptions than previous layers.

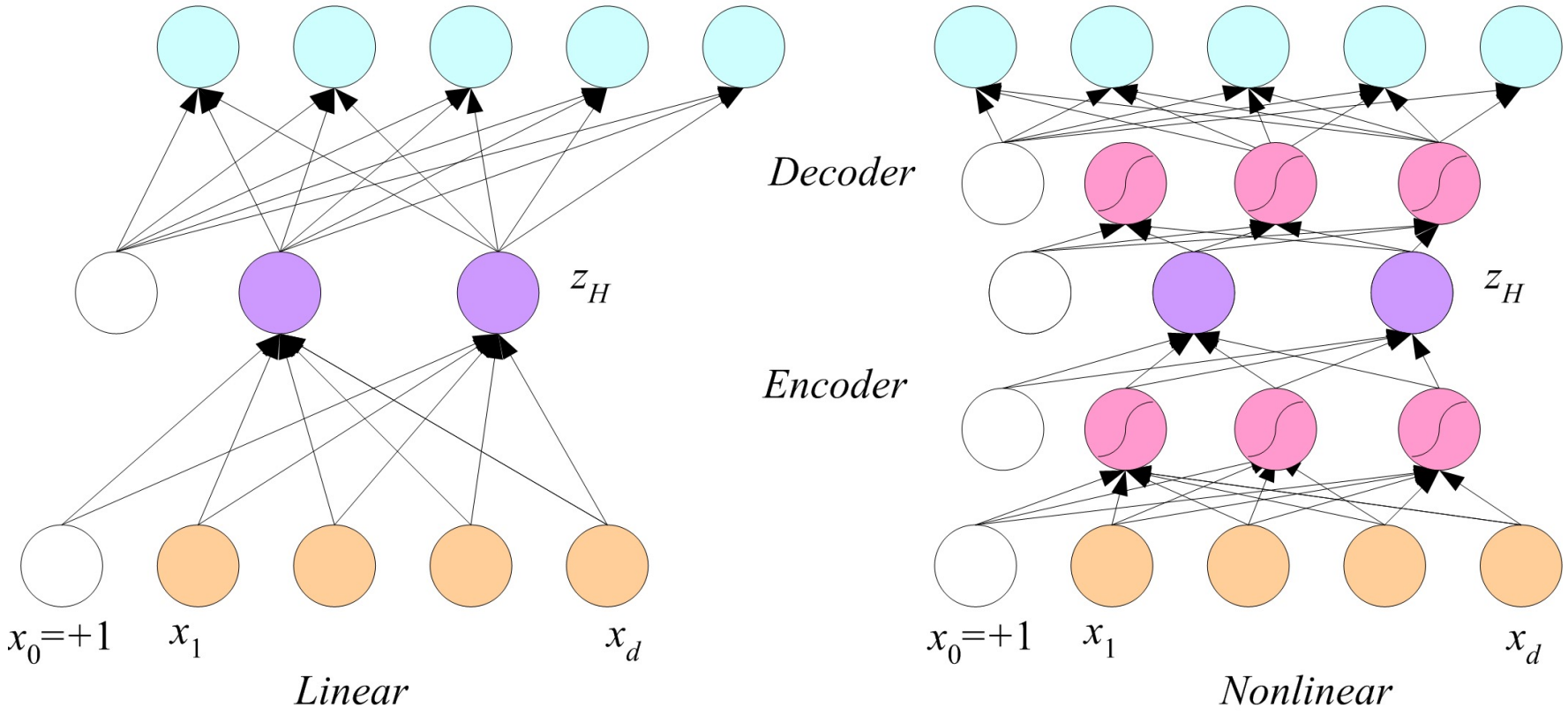
- A layer can instead have fewer nodes
- This ***reduces the dimensionality*** of the new feature space
- It forces the network to compress the internal representation of the problem

In practice, for many problems a combination of varying the dimensionally of the multiple hidden layers leads to a better performance

One application of dimensionality reduction is reconstruction

- The input to the network is “encoded” and then “decoded”
- This is what happens in networks used to “invent” new images
  - For example Google’s “Dream Generator”

Dimensionality Reduction



Summary

- Perceptrons
  - Processing & Training
  - Multiple perceptrons for classification
- Multi-Layer Perceptrons (MLP)
  - Feed-Forward processing
  - Training via Back-Propagation
  - Hypothesis Nature
  - Strengths & Weaknesses
- Extensions